

ITERATIVEDFS(s):

PUSH(s)

while the stack is not empty

$v \leftarrow \text{POP}$

if v is unmarked

mark v

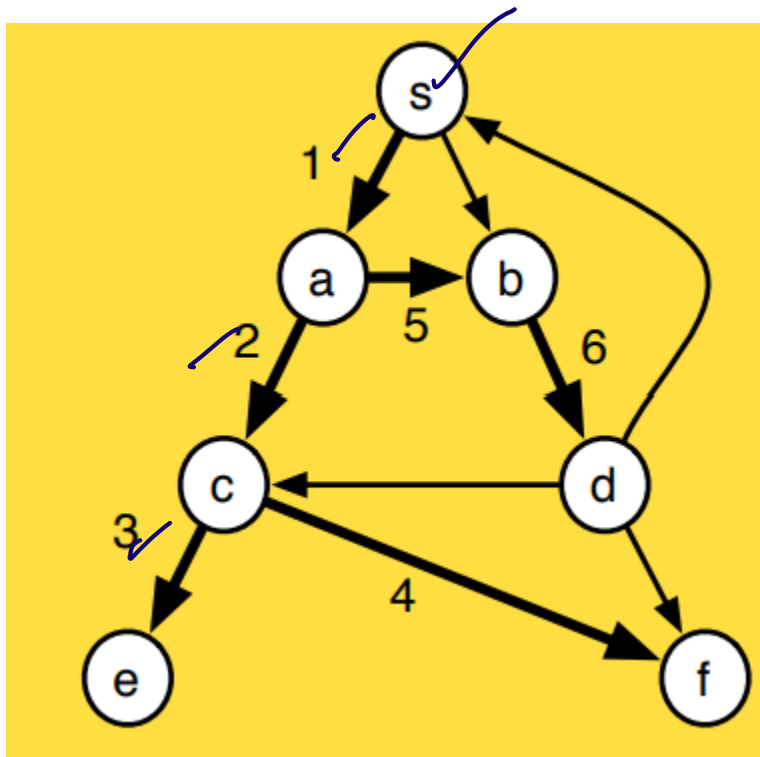
for each edge vw

PUSH(w)

~
Iterative
Recursive

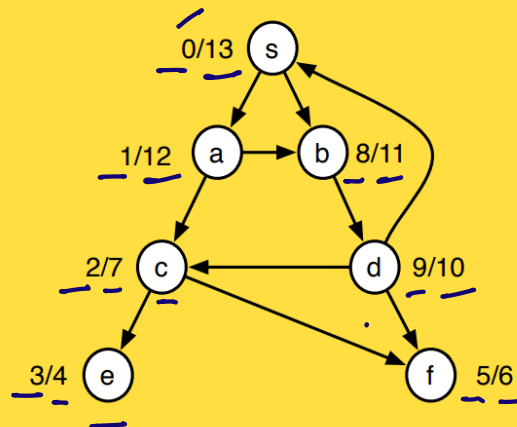
The complexity of performing a DFS is $O(V + E)$ as the total work we perform per node v is $O(\text{outdeg}(v))$ and

$$\sum_{v \in V} \text{outdeg}(v) = E$$



7

Example 11.5. A graph and its DFS numbers illustrated; t_1/t_2 denotes the timestamps showing when the vertex gets its white (discovered) and red pebble (finished) respectively.



DFS numbers have some interesting properties.

Exercise 11.6. Can you determine by just using the DFS numbers of a graph whether a vertex is an ancestor or a descendant of another vertex?

Definition 11.7. We call an edge (u, v) a tree edge if v receives its white pebble when the edge (u, v) was traversed. Tree edges define the DFS tree.

The rest of the edges in the graph, which are non-tree edges, can further be classified as back edges, forward edges, and cross edges.

- A non-tree edge (u, v) is a back edge if v is an ancestor of u in the DFS tree.
- A non-tree edge (u, v) is a forward edge if v is a descendant of u in the DFS tree.
- A non-tree edge (u, v) is a cross edge if v is neither an ancestor nor a descendant of u in the DFS tree.

Example 11.8. Tree edges (black), and non-tree edges (red, dashed) illustrated with the original graph and drawn as a tree.

